

Mohsin Iban Hossain

AIUB, Data Communication Notes

DATA COMMUNICATION

Table of Contents

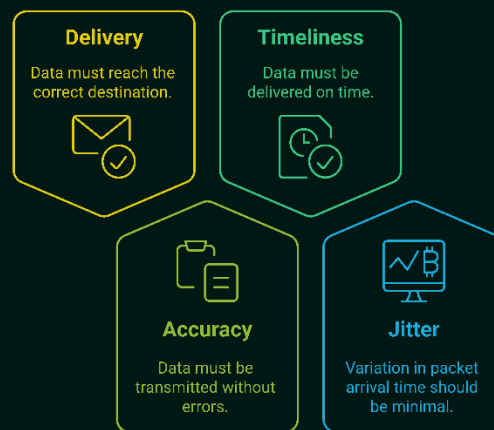
Sl. No	Topic	Pages
1	Introduction to Data Communication	3-14
2	Network Models	15-

INTRODUCTION TO DATA COMMUNICATION

Characteristics of Data Communication

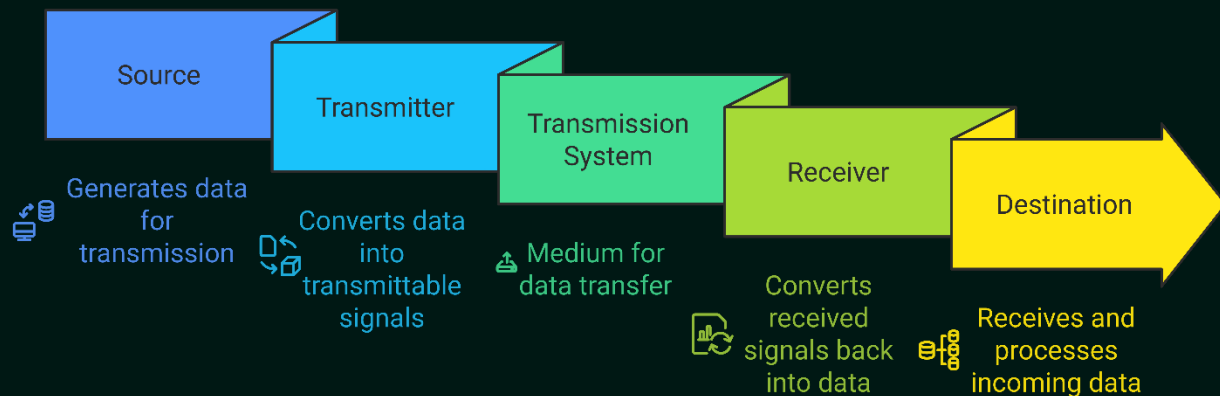
→ Data Communications:

- Exchange of data between devices via a transmission medium (e.g., wire cable).
- Requires hardware (physical equipment) and software (programs).
- **Effectiveness depends on:**



Key Elements of Data Communication

→ Elements of Data Communication:



A Data Communications Model

The Journey of a Digital Message

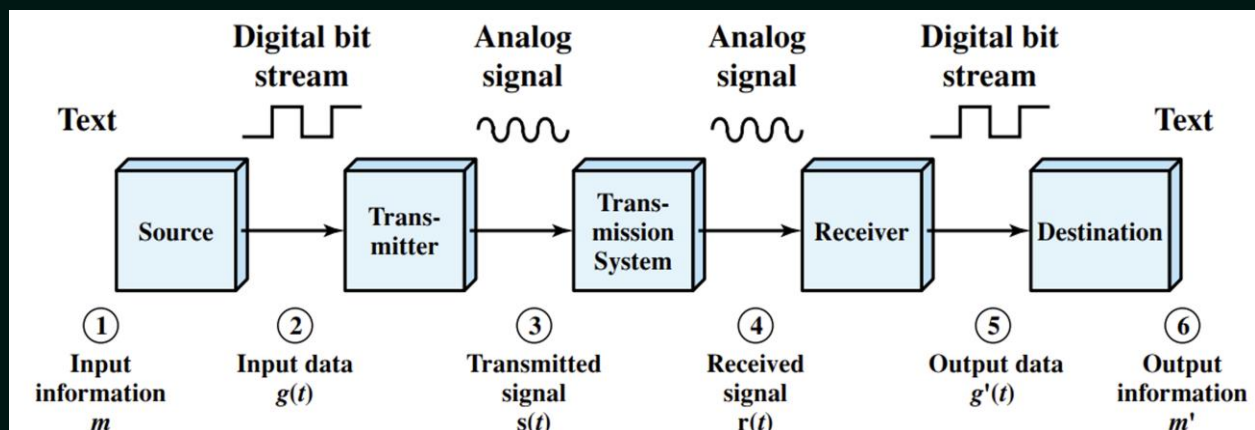
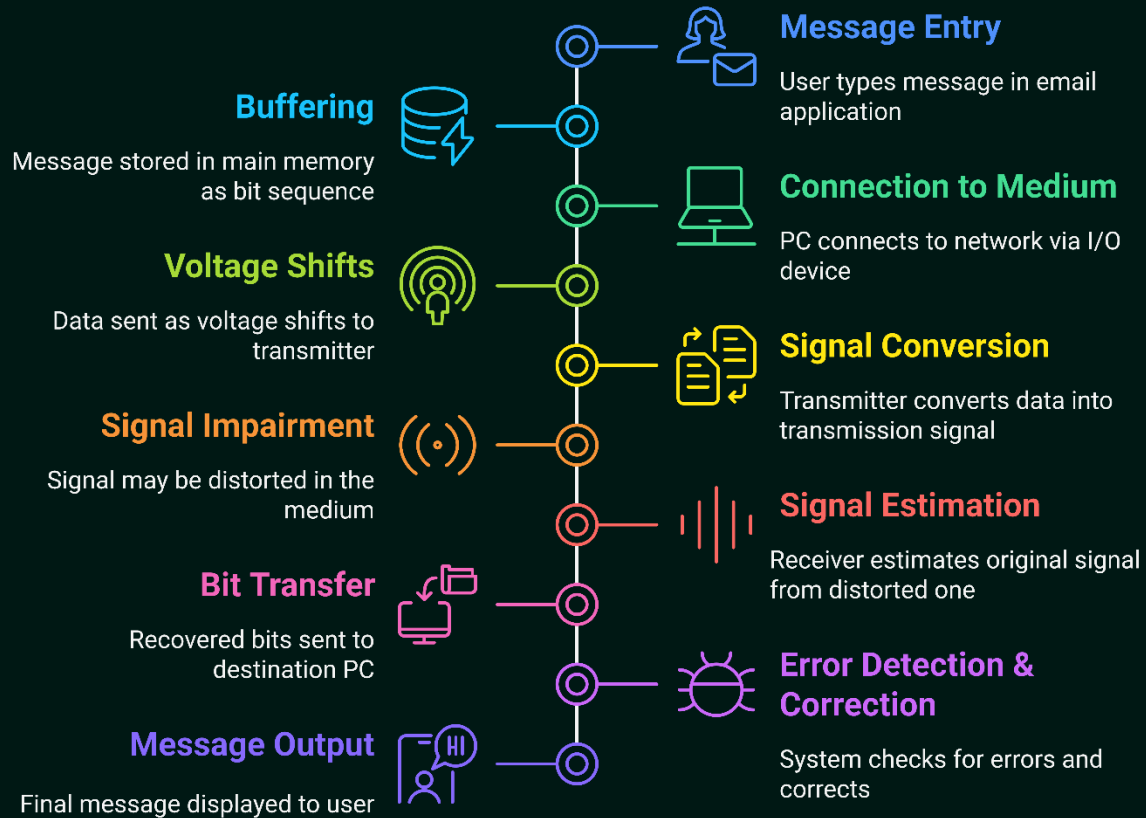


Figure: A Data Communications Model

Data Representation

➔ Binary Representation & Code Sets:

- A **bit** (binary digit) has two states: **0** and **1**.
- Computers require more symbols than a single bit can represent.
- Symbols include:
 - **52 letters** (uppercase & lowercase)
 - **10 numerals** (0–9)
 - **Punctuation & special characters**
 - **Control characters** (e.g., Carriage Return, Line Feed)

➔ Bit Groups & Symbol Representation:

- **5-bit code** → $2^5 = 32$ symbols.
- **8-bit code** → $2^8 = 256$ symbols.
- A **code set** assigns bit patterns to symbols.

⇒ Common Code Sets:

1. **ASCII (7-bit)** – American Standard Code for Information Interchange.
2. **EBCDIC (8-bit)** – Extended Binary-Coded-Decimal Interchange Code (IBM).

ASCII Code

➔ ASCII Code Set:

- **Most common & widely used** code set.
- **7-bit code** → **128 possible symbols** with defined meanings.

➔ ASCII Symbol Categories:

1. **96 Graphic Symbols** (Columns 2–7):
 - **94 Printable Characters** (letters, digits, punctuation, etc.).
 - **SPACE & DELETE** characters.
2. **32 Control Symbols** (Columns 0 & 1):
 - Used for **text formatting & device control** (e.g., Carriage Return, Line Feed).

7-Bit ASCII (American Standard Code for Information Interchange)								
	000...	001...	010...	011...	100...	101...	110...	111...
..0000	<i>NUL</i>	<i>DLE</i>	<i>SP</i>	0	@	P	'	p
..0001	<i>SOH</i>	<i>DC1</i>	!	1	A	Q	a	q
..0010	<i>STX</i>	<i>DC2</i>	"	2	B	R	b	r
..0011	<i>ETX</i>	<i>DC3</i>	#	3	C	S	c	s
..0100	<i>EOT</i>	<i>DC4</i>	\$	4	D	T	d	t
..0101	<i>ENQ</i>	<i>NAK</i>	%	5	E	U	e	u
..0110	<i>ACK</i>	<i>SYN</i>	&	6	F	V	f	v
..0111	<i>BEL</i>	<i>ETB</i>	,	7	G	W	g	w
..1000	<i>BS</i>	<i>CAN</i>	(	8	H	X	h	x
..1001	<i>HT</i>	<i>EM</i>	)	9	I	Y	i	y
..1010	<i>LF</i>	<i>SUB</i>	*	:	J	Z	j	z
..1011	<i>VT</i>	<i>ESC</i>	+	;	K	[	k	{
..1100	<i>FF</i>	<i>FS</i>	,	<	L	\	l	
..1101	<i>CR</i>	<i>GS</i>	-	=	M	]	m	}
..1110	<i>SO</i>	<i>RS</i>	.	>	N	^	n	~
..1111	<i>SI</i>	<i>US</i>	/	?	O	_	o	<i>DEL</i>

✂ Example 1:

➔ To determine the **binary representation** of a character from its **hexadecimal coordinate**:

1. The first hex digit represents the **row** (4 in this case).
2. The second hex digit represents the **column** (B in this case).
3. Convert both hex digits to **binary** separately:
 - 4 (Hex) → 100 (Binary)
 - B (Hex) → 1011 (Binary)
4. Combine both to get the **7-bit ASCII code**:
 - K (4B) → 100 1011 (Binary)

This method works for any ASCII character with a given **hex coordinate**.

🔗 Example 2:

⇒ ASCII Representation of "3P.bat":

⇒ Each character is represented using its 7-bit ASCII code, with the 8th bit set to 0:

Character	Hex Code	Binary (7-bit)	8-bit (MSB = 0)
3	33	011 0011	0011 0011
P	50	101 0000	0101 0000
.	2E	010 1110	0010 1110
b	62	110 0010	0110 0010
a	61	110 0001	0110 0001
t	74	111 0100	0111 0100

🔗 Final ASCII Binary Representation:

0011 0011 0101 0000 0010 1110 0110 0010 0110 0001 0111 0100

This is the 8-bit ASCII representation of "3P.bat".

Data Transmission

➔ Bytes & Data Transmission:

- **Byte:** A group of bits treated as a single unit during processing.
 - Typically, **8 bits long**, but can vary.
 - Example: **1001011 (ASCII for 'K')** is a byte with a defined meaning.
 - Some bytes **may not belong** to standard code sets.
- **Need for Data Exchange:**
 - Computers and terminals **exchange data, commands, and control information** in binary form.
- **Data Transmission:**
 - Movement of **bits** over a **physical medium** connecting digital devices.

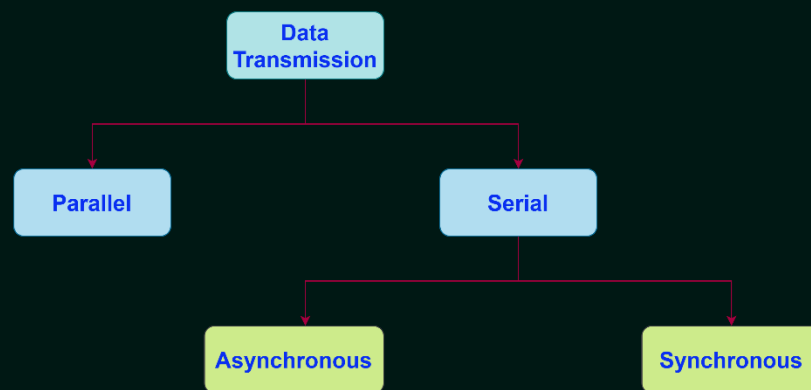
Types of Data Transmission:

1. Parallel Transmission

- Multiple bits are sent **simultaneously** over multiple channels.
- **Faster** but requires more wires.
- Used for **short-distance communication** (e.g., inside a computer, printer connections).

2. Serial Transmission

- Bits are sent **one after another** over a single channel.
- **Slower** but requires fewer wires, making it **cost-effective**.
- Used for **long-distance communication** (e.g., USB, Ethernet, Internet).



Serial Synchronous Transmission

- **Continuous transmission** of bits without start/stop bits or gaps.
- The **receiver** groups the bits based on timing.
- **Faster** and more **efficient** than asynchronous transmission.
- Used in **high-speed communication** (e.g., Ethernet)

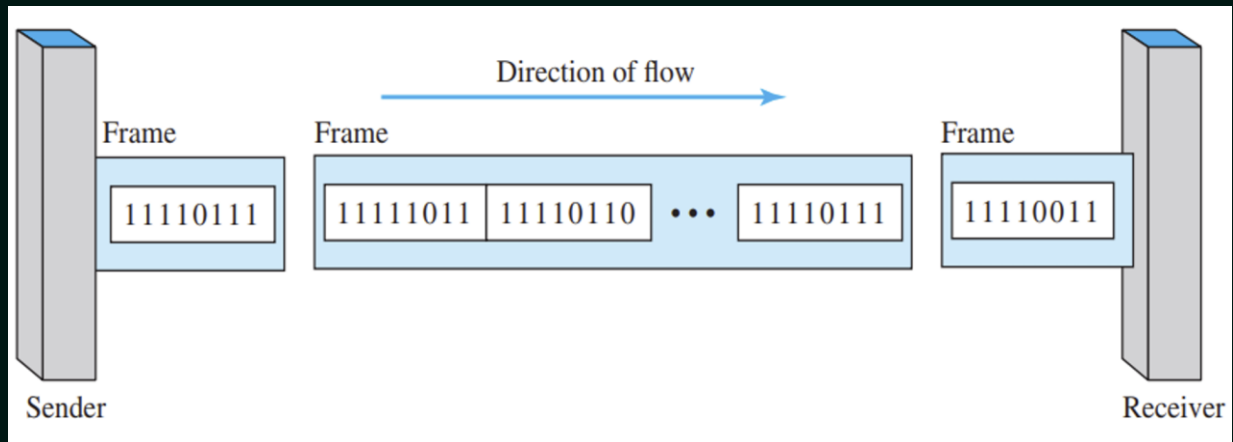


Figure: Synchronous transmission

✂ Example 1:

➔ Write the Synchronous bit transmission sequence of the message "HI".

Character	Binary (7-bit)	8-bit (MSB = 0)	After transmission
H	1001000	01001000	00010010
I	1001001	01001001	10010010

✂ Example 2:

➔ Write the Synchronous bit transmission sequence of the message "3P.bat".

Character	Binary (7-bit)	8-bit (MSB = 0)	After transmission
3	0110011	00110011	11001100
P	1010000	01010000	00001010
.	0101110	00101110	01110100
b	1100010	01100010	01000110
a	1100001	01100001	10000110
t	1110100	01110100	00101110

Serial Asynchronous Transmission

- Add **Start Bit (0)** at the beginning of each byte and **stop bits (1)** at the end.
- **Variable gaps** between bytes (also called framing).
- **Used for low-speed data transmission.**
- Each byte is sent **individually** with timing information provided by the start and stop bits.

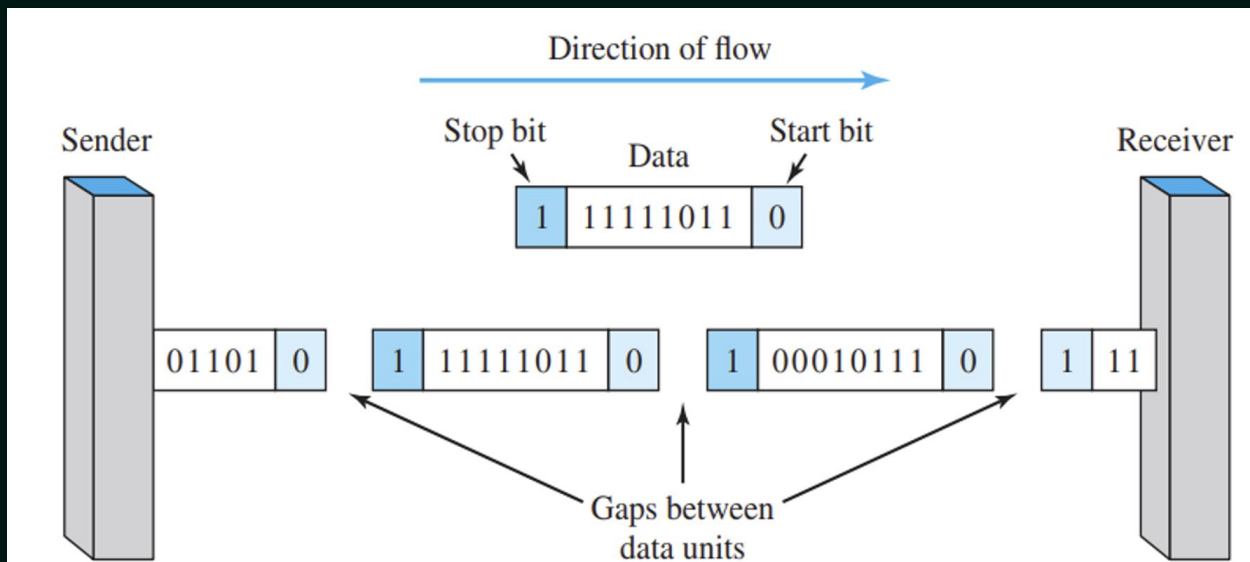


Figure: Asynchronous transmission

✂ Example 1:

➔ Write the Asynchronous bit transmission sequence of the message "HI".

Character	Binary (7-bit)	8-bit (MSB = 0)	Add Start & Stop Bit	After transmission
H	1001000	01001000	1010010000	0000100101
I	1001001	01001001	1010010010	0100100101

Bit Representation in Transmission

- **Electrical signals** are used to transmit bits over wires.
- **Two binary states** ("1" and "0") are represented by different voltage levels.

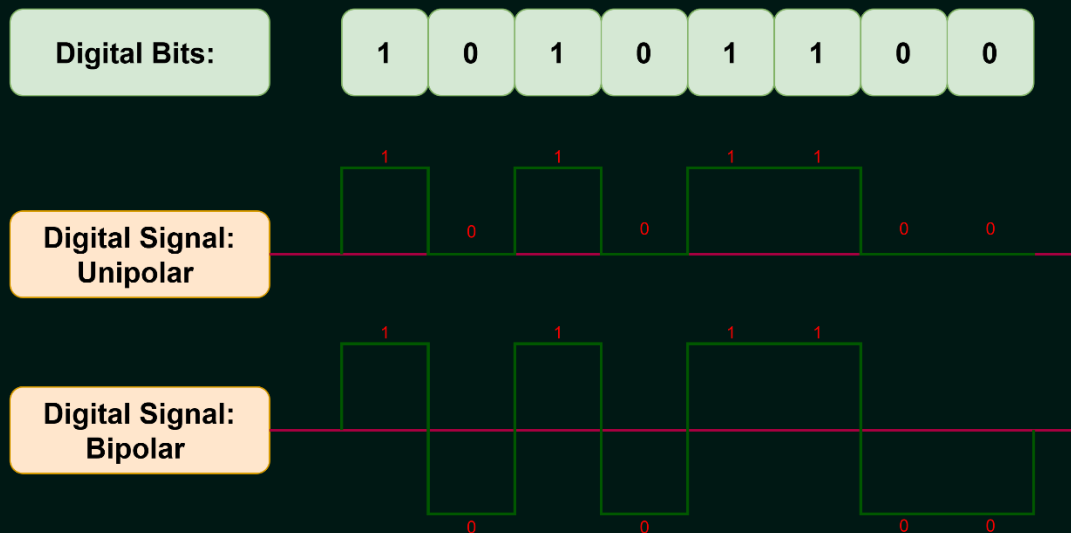
→ Types of Transmission:

1. Unipolar Transmission:

- One state (e.g., "0") is **0 volts**.
- The other state (e.g., "1") is a **positive voltage (+V volts)**.

2. Bipolar Transmission:

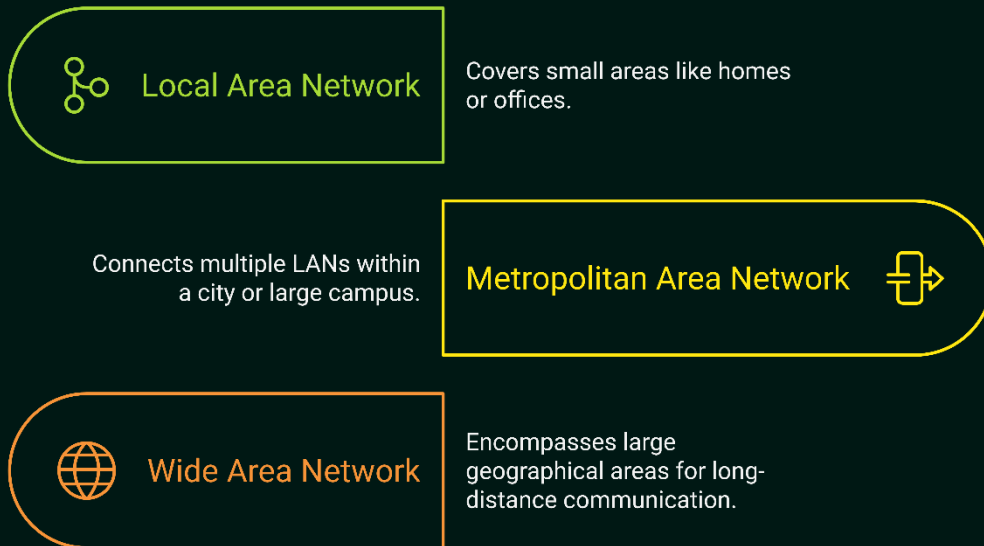
- "1" is represented by **+V volts**.
- "0" is represented by **-V volts**.
- More efficient in **reducing signal degradation** over long distances.



Bit Rate

- **Bit Rate (R):** Number of **bits transmitted per second**.
- **Formula:** $R = \frac{1}{t_p}$ where t_p is the **bit duration**.
- **Bit Duration $\neq$ Pulse Duration:**
 - A **pulse** can span multiple bit durations.
 - In some signal formats, **pulse duration** may be **half** of the bit duration.

Types of Networks



1. Local Area Network (LAN)

- Covers a **small area** (e.g., home, office, school).
- High **data transfer speed** and low latency.
- Example: **Wi-Fi, Ethernet** in a building.

2. Metropolitan Area Network (MAN)

- Covers a **city or large campus**.
- Connects multiple **LANs** within a **metropolitan area**.
- Example: **Cable TV networks, city-wide Wi-Fi**.

3. Wide Area Network (WAN)

- Covers **large geographical areas** (e.g., countries, continents).
- Slower than LAN/MAN but supports **long-distance communication**.
- Example: **The Internet, satellite networks**.

NETWORK MODELS

Layered Tasks

→ Protocol Layering:

- **Concept of Layers:** Used in daily life to **break down complex tasks** into **smaller, manageable steps**.
- **Why Use Layers?**
 - **Simplifies network design** by handling specific tasks at each layer.
 - **Enhances troubleshooting** since issues can be isolated to a single layer.
 - **Promotes compatibility** between different systems.
- **Example:** The **OSI model** and **TCP/IP model** use layering to structure network communication.

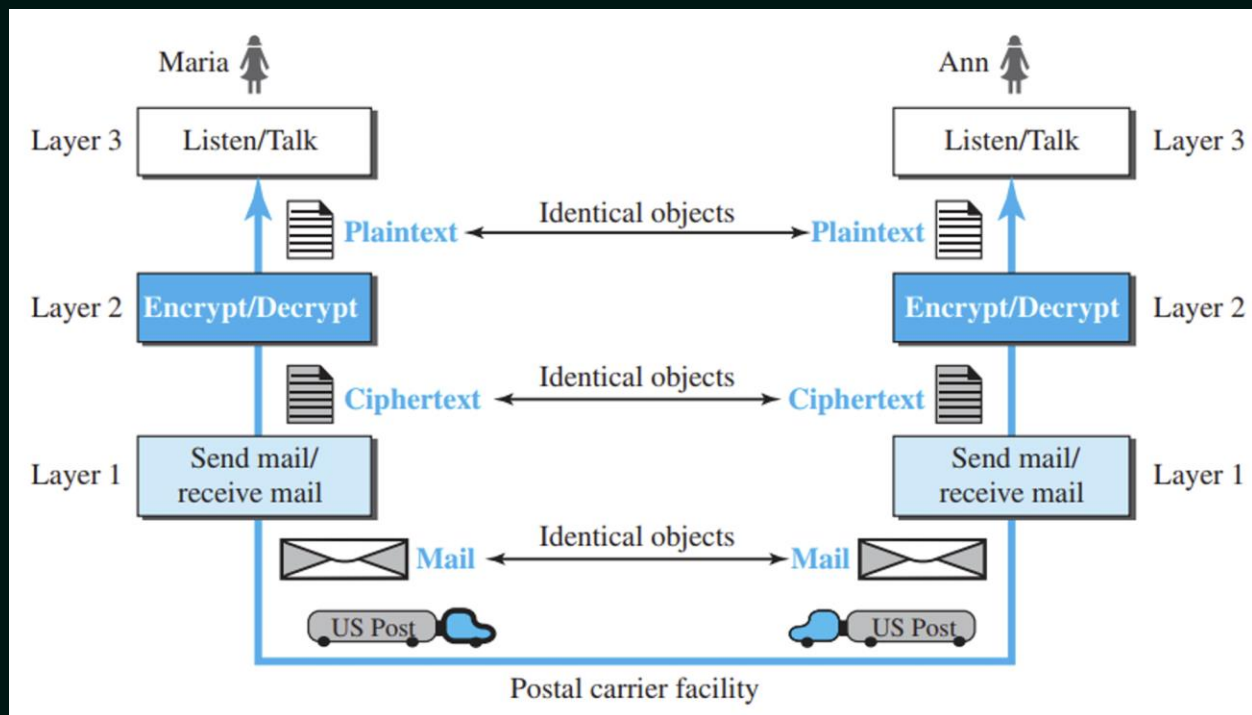


Fig: Tasks involved in sending a message in multiple layer

Importance of Layering

→ Advantages of Protocol Layering:

1. Troubleshooting

- Each layer has specific **protocols, actions, and data**.
- Issues can be **pinpointed** to the layer responsible for the problem.

2. Standards & Interoperability

- Provides a **structured guideline** for different vendors.
- Ensures that **devices from different manufacturers** can communicate seamlessly.

3. Flexibility & Change Management

- **Changes in one layer** have **minimal impact** on others.
- Prevents disruptions across the entire system.

Introduction to OSI Model

→ OSI Model – Seven Layers

The **Open Systems Interconnection (OSI) Model** is an **abstract framework** for network communication, developed by **ISO in 1978**. It consists of **seven layers**, each handling specific network functions.

OSI Seven Layers (Top to Bottom):

1. **Application Layer** – User interaction (e.g., web browsers, email).
2. **Presentation Layer** – Data formatting, encryption, compression.
3. **Session Layer** – Manages connections (start, maintain, terminate).

4. **Transport Layer** – Ensures reliable data delivery (TCP, UDP).
5. **Network Layer** – Handles routing and addressing (IP, routers).
6. **Data Link Layer** – Manages data frames and MAC addressing.
7. **Physical Layer** – Deals with actual transmission (cables, signals).

OSI Model	
Data unit	Layer
Data	7. Application
	6. Presentation
	5. Session
Segments	4. Transport
Packet	3. Network
Frame	2. Data Link
Bit	1. Physical

Application Layer

➔ Application Layer (Layer 7) – OSI Model

- **Closest to the end user**, directly interacting with software applications.
- Provides **network services** like web browsing, file transfer, and email communication.

Examples of Application Layer Protocols:

1. **HTTP (Hypertext Transfer Protocol)** – Used for **web browsing** (e.g., accessing websites).
2. **FTP (File Transfer Protocol)** – Facilitates **file transfers** between computers.
3. **SMTP (Simple Mail Transfer Protocol)** – Handles **email transmission**.

This layer ensures seamless communication between **user applications and network services**.

Presentation Layer

→ Presentation Layer (Layer 6) – OSI Model

- **Role:** Ensures **interoperability** between different encoding methods used by computers.
- **Responsibilities:**
 - Translates data between the **network format** and the **computer format**.
 - Handles **data representation differences**, such as **encryption** and **compression**.

Session Layer

→ Session Layer (Layer 5) – OSI Model

- **Controls dialogue** between two end systems (manages sessions).
- Defines how to **start, control, and end** conversations.
- Handles **log-on** and **password validation**.
- Responsible for **terminating** the connection.

Transport Layer

→ Transport Layer (Layer 4) – OSI Model

- **Purpose:** Provides reliable data exchange between processes on different computers.
- **Responsibilities:**
 - Ensures **error-free** data delivery.
 - Ensures data is delivered in **sequence**.

- Prevents **data loss** or **duplication**.
- Provides **connectionless** or **connection-oriented services**.
- Manages **connection establishment** and termination.
- **Multiplexes** multiple connections over a single channel.

Network Layer

→ Network Layer (Layer 3) – OSI Model

- **Routing:** Determines the **best route** for data to travel from source to destination.
- **Logical Addressing:**
 - Defines an **addressing scheme** (e.g., **IP addresses**) to uniquely identify devices on the internetwork.
 - Places the **sender** and **receiver's IP addresses** in the header to distinguish devices uniquely.

Data Link Layer

→ Data Link Layer (Layer 2) – OSI Model

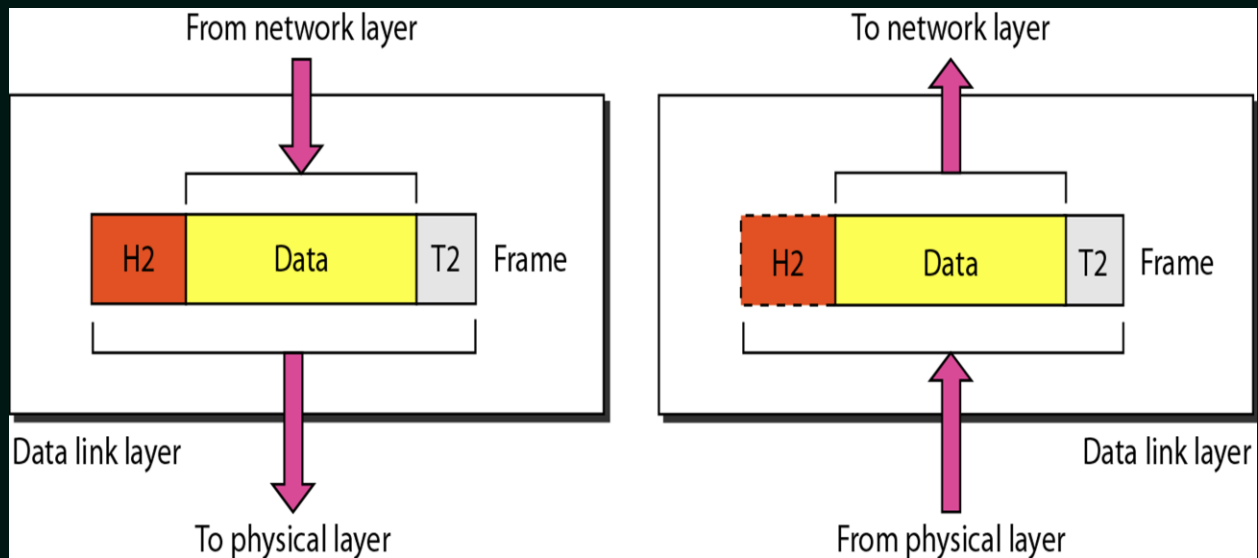
- **Function:** Ensures **node-to-node** error-free data transfer over the **physical layer**.
- **Sub-layers:**
 1. **MAC (Media Access Control):**
 - Manages how data is **transferred over the media** (e.g., CSMA/CD, CSMA/CA).
 2. **LLC (Logical Link Control):**
 - Controls **synchronization, flow control, and error checking** functions.

⇒ Services of Data-Link Layer:

- **Framing:** Divides the bit stream into **manageable data units** called **frames**.
- **Physical Addressing:** Adds a **header** to define the physical address (MAC address) of the sender/receiver.

- **Flow Control:** Ensures the receiver isn't overwhelmed by controlling the rate of data transfer.
- **Error Control:** Provides **reliability** by detecting and retransmitting lost or damaged frames.
- **Access Control:** Determines **which device controls the link** when multiple devices share the same connection.

➔ Data Link Layer - Frame:



The data link layer is responsible for moving frames from one hop (node) to the next

Physical Layer

➔ Physical Layer (Layer 1) – OSI Model

- **Bit Synchronization:** Provides synchronization of bits using a **clock** to control both sender and receiver.
- **Bit Rate Control:** Defines the **transmission rate** (bits per second).
- **Physical Topologies:** Specifies the physical arrangement of devices (e.g., **bus**, **star**, **mesh** topologies).
- **Transmission Modes:** Defines how data flows between devices:
 - **Simplex:** One-way transmission.
 - **Half-Duplex:** Two-way transmission, but not simultaneously.
 - **Full-Duplex:** Two-way transmission simultaneously.

Summary

7. Application	• Provides a user interface
6. Presentation	• Presents Data • Handles encryption and decryption
5. Session	• Maintains distinction between data of separate applications • Provides dialog control between hosts
4. Transport	• Provides End-to-End connections • Provides reliable or unreliable delivery and flow control
3. Network	• Provides Logical Addressing • Provides Path determination using logical addressing
2. Data Link	• Provides media access and physical addressing
1. Physical	• Converts digital data so that it can be sent over the physical medium • Moves data between hosts

